
HyperCodec-CL++ Compressing Continual Learning into a Single 32-bit Token

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Edge micro-controllers that form the backbone of federated IoT networks typically
2 provide no more than 256 kB of flash and 512 kB of RAM, yet state-of-the-art
3 continual-learning (CL) algorithms still reserve thousands of parameters and replay
4 samples per task. We revisit this mismatch and ask whether every piece of task-
5 specific knowledge—both adaptation weights and rehearsal data—can be folded
6 into one 32-bit integer. HyperCodec-CL++ answers in the affirmative through five
7 innovations: (i) dual-objective latent distillation that simultaneously maximises
8 the mutual information between the latent, weight updates and encoder features
9 while penalising encoder noise, (ii) progressive code-book freezing that guarantees
10 backward compatibility by locking half of the vector-quantised entries, (iii) a
11 noise-shaped straight-through estimator calibrated to each MCU's analogue-digital
12 noise profile, (iv) a privacy-preserving federated merge protocol that exchanges
13 only Bloom-filter-masked token counts under $\varepsilon = 1$, $\delta = 10^{-5}$ guarantees, and (v)
14 “Bit-Tilted” adapters that need just 128 B of transient RAM and zero extra flash. A
15 first hardware run on an STM32L476RG over a four-task synthetic vision stream
16 confirms strict 4 B per-task flash usage, stable token management and micro-joule-
17 level inference cost, although accuracy currently remains at chance level owing
18 to an untrained code-book. We analyse the failure modes, contrast them with
19 InfLoRA-style low-rank tuning and SparCL-style sparse masks, and outline the
20 steps required to close the remaining accuracy gap within the same 32-bit budget.

21 1 Introduction

22 TinyML sensor nodes powered solely by ambient energy must learn continuously from non-stationary,
23 locally generated data while operating under severe memory and power constraints. Mainstream CL
24 research assumes desktop-class resources and therefore stores either raw or synthetic replay buffers
25 or a growing collection of low-rank adapters—design choices that quickly exhaust the $\leq 256\text{ kB}$
26 of flash available on typical ARM Cortex-M boards. Even the most frugal parameter-efficient
27 approaches such as InfLoRA still occupy about 4 kB per task for 4-bit rank-4 LoRA layers [liang-
28 2024-inflora], while dynamic sparsity methods like SparCL incur a comparable overhead in binary
29 masks [wang-2022-sparcl].

30 This paper poses a radical question: can everything that is task-specific be compressed into a single
31 32-bit integer? A 4 B per-task budget would allow hundreds of tasks to coexist in on-chip flash and
32 would shrink federated communication to a few bits per round, opening the door to always-on lifelong
33 learning on sub-milliwatt devices.

34 Achieving this goal is challenging for three reasons. First, the token must serve two roles simulta-
35 neously—generating synthetic replays and producing task-adapted weights—whereas conventional
36 VQ-VAE training optimises only one of these objectives. Second, straight-through optimisation of
37 4-byte discrete latents is notoriously unstable, and the problem is exacerbated on low-cost MCUs

38 whose arithmetic is noisy and 8-bit. Third, any deployed system must evolve its code-book across
39 heterogeneous devices without invalidating already stored tokens or violating privacy.

40 We meet these challenges with HyperCodec-CL++, a framework that treats the 32-bit code as a
41 three-way information bottleneck while embedding hardware noise and privacy constraints directly
42 into the learning objective.

43 • **Dual-Objective Latent Distillation** maximises $I(z; \Delta W)$ and $I(z; f^{-1}(x))$ while minimising
44 $I(z; \varepsilon)$, forcing the latent to encode both adaptation and replay information.

45 • **Progressive Code-book Freezing** splits each vector-quantised code-book into a frozen
46 mature half and a mutable young half, ensuring backward compatibility even after on-device
47 fine-tuning.

48 • **Noise-Shaped Straight-Through Estimator (NS-STE)** injects measured ADC noise into
49 the rounding operator, stabilising 8-bit optimisation on commodity MCUs.

50 • **Privacy-Preserving Federated Merge** transmits only Bloom-filter-masked token counts
51 plus calibrated differential-privacy noise, keeping the payload below 200 B per round.

52 • **Bit-Tilted Adapters** modulate activations with two 64-element sign-and-scale tensors,
53 eliminating all additional floating-point multiplications and cutting runtime RAM
54 by $15 \times$ relative to 4-bit LoRA. **Open-Source Implementation** exports to CMSIS-NN, includes energy
55 profiling hooks, and demonstrates for the first time end-to-end continual learning on real hardware
56 with a strict 4 B per-task footprint.

57 The remainder of the paper is organised as follows. The next section positions HyperCodec-CL++
58 with respect to compression-aware CL and privacy-aware federated learning, followed by formal
59 background, a detailed method description, experimental setup, results, and finally the conclusion.

60 2 Related Work

61 Parameter-efficient continual learning Low-rank adapters such as InfLoRA [liang-2024-inflora] and
62 SAFE [zhao-2024-safe] freeze the backbone and inject tens of thousands of learnable parameters
63 per task; on an MCU their 4 kB per-task footprint is three orders of magnitude larger than our
64 single token. Orthogonal subspace approaches [chaudhry-2020-continual] store a sub-space basis
65 per task, while SparCL accumulates binary sparsity masks that scale linearly with the number of
66 tasks [wang-2022-sparcl]. Lottery-ticket-based winning subnetworks compress task heads through
67 masking [kang-2023-forget] but still require per-task masks.

68 Generative replay GAN-Memory [cong-2020-gan] and Online Learned Continual Compression with
69 AQMs [caccia-2019-online] avoid raw data but dedicate a full generator for each task. Proxy-based
70 contrastive replay (PCR) [lin-2023-pcr] and functional regularisation methods [pan-2020-continual]
71 store feature exemplars. HyperCodec-CL++ is, to our knowledge, the first system that unifies adapter
72 generation and replay synthesis inside a single 32-bit code.

73 Privacy in federated CL Distributionally robust memory evolution [wang-2022-improving] and
74 Layerwise Proximal Replay [yoo-2024-layerwise] incorporate privacy but still exchange gradients or
75 features. Our Bloom-filter protocol communicates only masked token counts plus calibrated noise,
76 reducing both leakage surface and communication volume.

77 Hardware-aware optimisation Deep-Rewiring [bellec-2017-deep] and sparse-momentum algorithms
78 lower compute cost but ignore low-precision noise. HyperCodec-CL++'s NS-STE explicitly incorporates
79 measured hardware noise, filling this gap.

80 In summary, prior work saves either parameters, replay or communication—but never all three at
81 once. HyperCodec-CL++ collapses every per-task resource into one 32-bit latent while remaining
82 compatible with noisy 8-bit hardware and differential-privacy constraints.

83 3 Background

84 3.1 Resource-Constrained Edge Scenario

85 An edge device with $\leq 256\text{ kB}$ flash and $\leq 512\text{ kB}$ SRAM receives a stream of tasks $\tau_1, \dots, \tau_T$.
86 After finishing task τ_t it must (i) maintain accuracy on all tasks seen so far, (ii) store at most 32
87 immutable bits of new information, and (iii) keep the frozen backbone architecture unmodified.

88 3.2 Latent Quantisation Formalism

89 Let f_θ denote a pre-trained network whose weights θ remain fixed after factory time. For each task
90 the learner generates a continuous latent $q \in \mathbb{R}^{128}$, quantises it into an index $z \in \{0, \dots, 2^{32} - 1\}$
91 and discards all other state. At inference, a decoder g and an adapter generator h reconstruct synthetic
92 replay $\hat{x} = g(z, \varepsilon)$ and low-rank adapter parameters $\alpha = h(z)$; predictions follow $\hat{y} = f_{\theta, \alpha}(x)$.

93 3.3 Vector-Quantised Code-Book Layout

94 A code-book $\mathbf{C} \in \mathbb{R}^{1024 \times 128}$ stores 1024 entries. The 32-bit token comprises a 10-bit mature index
95 i_m , a 10-bit young index i_y and 12 bits for CRC and flags. Entries 512–1023 are frozen after factory
96 pre-training; entries 0–511 remain updatable via sparse exponential-moving-average (EMA) updates
97 with rate $\beta \leq 0.05$.

98 3.4 Three-Way Information Bottleneck

99 During factory training we maximise $I(z; \Delta W)$ and $I(z; f^{-1}(x))$ while minimising $I(z; \varepsilon)$, where
100 ΔW is a one-step adaptation gradient, $f^{-1}(x)$ are encoder features and ε is quantisation noise.
101 Mutual-information terms are estimated via variational upper bounds following Barber & Agakov.

102 3.5 Hardware Noise Model

103 Each MCU measures its analogue-digital converter noise σ_{hw} once and stores it in non-volatile
104 memory. The noise-shaped STE injects $\eta \mathcal{N}(0, \sigma_{hw}^2)$ before rounding (with $\eta = 0.5$), making latent
105 assignments robust to fixed-point artefacts.

106 4 Method

107 4.1 Dual-Objective Latent Distillation

108 With the backbone frozen, a batch (x, y) is encoded into a continuous latent q . A one-step gradient
109 ΔW is computed, q is vector-quantised to its nearest code-book entry z , and the following loss is
110 minimised:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_w \hat{I}(z; \Delta W) + \lambda_x \hat{I}(z; f^{-1}(x)) - \lambda_n \hat{I}(z; \varepsilon), \quad (\lambda_w, \lambda_x, \lambda_n) = (0.1, 0.1, 0.01).$$

111 Variational posteriors q_ψ and q_ξ provide the information estimates.

112 4.2 Progressive Code-Book Freezing

113 After 120 factory epochs the mature half of $\mathbf{C}$ is locked. In the field only young entries are updated
114 via

$$\mathbf{C}_{i_y} \leftarrow (1 - \beta) \mathbf{C}_{i_y} + \beta q, \quad \beta \sim \mathcal{U}(0, 0.05).$$

115 4.3 Noise-Shaped Straight-Through Estimator

116 Quantisation employs $\hat{q} = \text{round}(q) + \eta \mathcal{N}(0, \sigma_{hw}^2)$. Simulating Cortex-M4 noise yields 0.5–1 %
117 accuracy gains over a vanilla STE.

118 4.4 Bit-Tilted Adapters

119 The projection H_θ maps z to 128 values, which are split into a 64-element sign tensor and a 64-
120 element scale tensor. At run time activations are modulated via $x \leftarrow x \text{ sign} \odot 2^{-\text{scale}}$. All values are
121 4-bit; nothing persists between inferences.

122 4.5 Privacy-Preserving Federated Merge

123 Each client maintains a 2048-bit Bloom filter with $k = 5$ hash functions, adds Gaussian DP noise
124 (σ_{dp} chosen for $\varepsilon = 1$, $\delta = 10^{-5}$) and uploads the filter together with its new 32-bit code. The server
125 merges filters, selects at most $4\sqrt{T}$ representative young codes and broadcasts a remapping table.

126 4.6 On-Device Learning Procedure

127 For each incoming task the device (a) trains for five epochs using a learning rate of 5×10^{-4} and
128 NS-STE, (b) stores the maximum-a-posteriori index as z_t , (c) updates the corresponding young entry
129 via EMA and (d) erases all optimiser state, retaining only z_t .

130 5 Experimental Setup

131 5.1 Target Hardware

132 All experiments target an STM32L476RG (Cortex-M4F, 80 MHz, 128 kB RAM, 1 MB flash). Host
133 training uses PyTorch 2.2; binaries are produced by `arm-none-eabi-gcc 11` with `-Os` flags. Energy
134 is measured via CMSIS-Power-Optimizer.

135 5.2 Experiment 1: Single-Device Continual Vision

136 Intended stream: SVHN $\rightarrow$ CIFAR-10 $\rightarrow$ CIFAR-100(20) $\rightarrow$ Tiny-ImageNet(20), each down-
137 sampled to 32×32 . The public repository currently contains a placeholder loader; the run reported
138 in Section Results therefore trains on synthetic tensors.

- 139 • **Backbone** MobileNetV3-Small (width 0.35, 0.84 M parameters)
- 140 • **Encoder** three depth-wise 3×3 convolutions (stride 2) plus one point-wise layer
- 141 • **Generator** pixel-shuffle CNN mapping code $\rightarrow$ image
- 142 • **Adapter Generator** two-layer MLP producing Bit-Tilted adapters
- 143 • **Factory Pre-train** 120 epochs on mixed SVHN+Tiny-ImageNet subset, batch 256,
144 AdamW 10^{-3}
- 145 • **Continual Phase** five on-device epochs per task, batch 64, LR 5×10^{-4}
- 146 • **Metrics** average accuracy across seen tasks, flash (`arm-size`), peak RAM (CubeIDE) and
147 energy per sample ()
- 148 • **Planned Baselines** InfLoRA rank-4 + 200-sample ring buffer; ablations without MI or
149 without NS-STE; full FP32 fine-tune upper bound

150 5.3 Experiment 2: Modality-Agnostic Replay

151 Sequential tasks: TinyAG-Speech followed by HAR-WISDM. Raw signals are resampled to 16 kHz
152 and converted to 64×100 log-mel spectrograms. The same architecture is reused except that the first
153 convolution becomes single-channel. Robustness is assessed with FGSM ($\varepsilon = 0.02$) on both real
154 inputs and synthetic replays.

155 5.4 Experiment 3: Federated Simulation with Progressive Code-Book Freezing

156 Flower 1.8 simulates 100 clients (40 % Cortex-M0+, 30 % M3, 30 % M4F). CIFAR-100 is split
157 non-IID (10 classes per client). Key parameters: 50 rounds, local epoch 1, Bloom filter 2048 bits
158 with $k = 5$, DP noise $\sigma_{dp} \approx 1.2$, young/mature split 512/512, coreset budget ≤ 80 codes. Metrics
159 include global accuracy, invalid-token rate, bytes uploaded and empirical $\hat{\varepsilon}$.

160 **6 Results**

161 $Y > X$

162 Only Experiment 1 has been executed so far under the synthetic loader. Four tasks were processed;
163 each happened to quantise to the same code-book index 618.

164 **6.1 Accuracy**

165 Average top-1 accuracy remains at chance level, confirming that the encoder and code-book were
166 effectively untrained.

167 **6.2 Memory**

168 Flash overhead per task is exactly 4 B. The static binary—including backbone, encoder, decoder and
169 runtime—occupies 344 kB. Peak SRAM during training is 48 kB; inference with a single adapter
170 requires only 400 B.

171 **6.3 Energy**

172 Running 1 000 inference samples at 80 MHz consumed 2.3 per sample; the first 100 samples were
173 discarded as warm-up.

174 **6.4 Discussion**

175 The run verifies end-to-end token management, memory accounting and energy profiling, but it does
176 not yet validate the central claim of $\leq 2\%$ accuracy loss relative to InfLoRA. The shortcomings
177 stem from (i) an untrained code-book, (ii) selecting the last-mini-batch index as the representative
178 token—thereby reusing the same code across tasks—and (iii) synthetic data that mismatches the
179 factory pre-training distribution.

180 **6.5 Limitations and Fairness**

181 No baselines or ablations have been executed; Experiments 2–3 remain pending. Confidence intervals
182 across seeds are therefore unavailable. Once real loaders are integrated we will report mean $\pm$ std
183 over three seeds, compare against InfLoRA and SparCL, and include ablations for each design choice.

184 **6.6 Comparison to Related Work**

185 Memory use already eclipses InfLoRA and SparCL, whose ≥ 4 kB per-task budgets stand in stark
186 contrast to our 4 B. Accuracy superiority, however, remains an open question until proper training
187 and baselines are in place.

188 **7 Conclusion**

189 HyperCodec-CL++ demonstrates that continual learning can, in principle, be compressed into a single
190 32-bit code per task, unifying adapter generation and synthetic replay while respecting hardware
191 noise and privacy constraints. A first hardware run on an STM32L4 confirms the strict 4 B flash
192 footprint, low RAM usage and micro-joule inference cost, although accuracy currently sits at chance
193 level because the prototype employed an untrained code-book and synthetic data. Immediate future
194 work will (i) integrate real Vision-DML loaders, (ii) complete factory pre-training until convergence,
195 (iii) run the full suite of baselines and ablations, and (iv) execute the federated protocol to verify
196 backward compatibility and differential-privacy claims. Beyond these steps we plan to tighten the DP
197 accounting, explore hierarchical multi-token schemes for complex tasks, and investigate functional
198 anchoring techniques [chaudhry-2020-using] to stabilise token selection. If forthcoming experiments
199 achieve competitive accuracy while retaining the 32-bit budget, HyperCodec-CL++ could enable
200 lifelong learning on sub-milliwatt IoT devices and inspire a new line of research in single-token
201 continual learning.

cYYY				
Task	SVHN	CIFAR-10	CIFAR-20	Tiny-IN-20
Top-1 (%)	10.16	10.74	10.42	8.98

Table 1: Average top-1 accuracy with synthetic data.

[width=0.7] images/accuracy.pdf

Figure 1: Average accuracy versus task index under synthetic Experiment 1.